

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 - Searching



Jonathan Andrew Wijaya

109082500208

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Algoritma pencarian adalah metode sistematis untuk menemukan suatu data tertentu dalam kumpulan data. Dalam pemrograman, algoritma pencarian digunakan untuk mencari elemen dengan nilai tertentu dalam struktur data seperti array atau list. Terdapat dua metode pencarian yang umum digunakan, yaitu Sequential Search dan Binary Search.

Sequential search atau pencarian berurutan adalah algoritma pencarian paling sederhana yang bekerja dengan memeriksa setiap elemen dalam array secara berurutan dari awal hingga akhir sampai data yang dicari ditemukan atau seluruh elemen telah diperiksa.

Binary search atau pencarian biner adalah algoritma pencarian yang lebih efisien dengan cara membagi area pencarian menjadi dua bagian secara berulang. Algoritma ini hanya dapat digunakan pada data yang sudah terurut (ascending atau descending).

B. Guided

1. Sequential search

```
package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string
```

```

for i := 0; i < len(arrBinatang); i++ {
    fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
    fmt.Scan(&arrBinatang[i])
}
fmt.Println()

var binatangCari string
fmt.Print("Masukkan nama binatang yang mau dicari : ")
fmt.Scan(&binatangCari)

var idxCari int
idxCari = seqSearch(arrBinatang, binatangCari)

if idxCari > -1 {
    fmt.Printf("Data %s ditemukan pada indeks ke-%d!",
binatangCari, idxCari)
} else if idxCari == -1 {
    fmt.Printf("Data %s tidak ditemukan!", binatangCari)
}
}

```

Output :

```

PS C:\Users\COLORFUL\Documents\ALPROTELKOM\SEMESTER 2> go run
Wijaya\Week11\Guided\Guided1\sequentialsearch.go
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : sapi
Masukkan data binatang indeks ke-2 : ayam
Masukkan data binatang indeks ke-3 : ikan
Masukkan data binatang indeks ke-4 : kambing

Masukkan nama binatang yang mau dicari : ayam
Data ayam ditemukan pada indeks ke-2!

```

Penjelasan:

Program ini digunakan untuk mencari nama binatang dalam sebuah array yang berisi 5 data. Program akan meminta pengguna memasukkan lima nama binatang, lalu meminta satu nama binatang yang ingin dicari. Pencarian dilakukan menggunakan metode sequential search, yaitu mengecek data satu

per satu dari indeks awal sampai akhir. Jika data ditemukan, program menampilkan indeks tempat data tersebut berada. Jika tidak ditemukan, program menampilkan pesan bahwa data tidak ada.

2. Binary Search

```
package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {

        // mencari index tengah
        tengah := (kiri + kanan) / 2

        // jika data ditemukan
        if data[tengah] == x {
            return tengah
        }

        // jika x lebih besar, cari ke kanan
        } else if data[tengah] < x {
            kiri = tengah + 1

        // jika x lebih kecil, cari ke kiri
        } else {
            kanan = tengah - 1
        }
    }

    // jika data tidak ditemukan
    return -1
}
```

```

}

func main() {

    var n int
    var cari int

    // input jumlah data
    fmt.Print("Jumlah data: ")
    fmt.Scan(&n)

    // membuat array sesuai jumlah data
    data := make([]int, n)

    // input isi array
    fmt.Println("Masukkan data terurut:")
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    // input angka yang dicari
    fmt.Print("Angka yang dicari: ")
    fmt.Scan(&cari)

    // memanggil fungsi binary search
    hasil := binarySearch(data, cari)

    // output hasil pencarian
    if hasil != -1 {
        fmt.Println("Data ditemukan di index:", hasil)
    } else {
        fmt.Println("Data tidak ditemukan")
    }
}

```

Output :

```

Jumlah data: 7
Masukkan data terurut:
10 23 35 47 52 68 91
Angka yang dicari: 47
Data ditemukan di index: 3

```

Penjelasan:

Program ini digunakan untuk mencari sebuah angka pada data yang sudah terurut menggunakan metode binary search. Program meminta pengguna memasukkan jumlah data, isi data secara terurut, dan angka yang ingin dicari. Fungsi `binarySearch` bekerja dengan menentukan batas kiri, batas kanan, dan indeks tengah, lalu membandingkan nilai tengah dengan angka yang dicari. Jika angka lebih besar dari nilai tengah, pencarian dilanjutkan ke bagian kanan. Jika lebih kecil, pencarian dilanjutkan ke bagian kiri. Jika data ditemukan, program menampilkan indeksnya, sedangkan jika tidak ditemukan program menampilkan pesan bahwa data tidak ditemukan.

3. [judul guided 3]

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
```

```

        kn = med - 1
    } else if nimDicari > arrData[med].nim {
        kr = med + 1
    } else {
        found = med
    }
}
return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06 pada indeks ke-%d\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

```

```

var idxHasilCariSeqSearch int
idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06, nimDicari)

fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
if idxHasilCariSeqSearch > -1 {
    fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
    fmt.Println("Berikut Detail Datanya : ")
    fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariSeqSearch].nama)
    fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariSeqSearch].nim)
    fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariSeqSearch].IPK)
} else if idxHasilCariSeqSearch == -1 {
    fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
}
fmt.Println()

var idxHasilCariBinSearch int
idxHasilCariBinSearch = binSearchStruct(mahasiswa06, nimDicari)

fmt.Println("== HASIL BINARY SEARCH ==")
if idxHasilCariBinSearch > -1 {
    fmt.Printf("Data NIM mahasiswa %d ditemukan pada array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
    fmt.Println("Berikut Detail Datanya : ")
    fmt.Printf("Nama : %s\n", mahasiswa06[idxHasilCariBinSearch].nama)
    fmt.Printf("NIM : %d\n", mahasiswa06[idxHasilCariBinSearch].nim)
    fmt.Printf("IPK : %.2f\n", mahasiswa06[idxHasilCariBinSearch].IPK)
} else if idxHasilCariBinSearch == -1 {
    fmt.Printf("Data NIM mahasiswa %d tidak ditemukan pada array!", nimDicari)
}
}

```

Output :


```

--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : Andi
Masukkan NIM : 101
Masukkan IPK : 3.55
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : Budi
Masukkan NIM : 102
Masukkan IPK : 3.67
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : Nala
Masukkan NIM : 103
Masukkan IPK : 3.7
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : Dina
Masukkan NIM : 104
Masukkan IPK : 3.22
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : Eka
Masukkan NIM : 105
Masukkan IPK : 3.88
-----

Masukkan NIM mahasiswa yang ingin dicari : 104

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 104 ditemukan pada array di indeks ke-3!
Berikut Detail Datanya :
Nama : Dina
NIM : 104
IPK : 3.22

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 104 ditemukan pada array di indeks ke-3!
Berikut Detail Datanya :
Nama : Dina
NIM : 104
IPK : 3.22

```

Penjelasan:

Program ini digunakan untuk mencari data mahasiswa berdasarkan NIM dalam array berisi 5 data mahasiswa. Setiap data mahasiswa memiliki nama, NIM, dan IPK. Program meminta pengguna memasukkan data mahasiswa secara terurut berdasarkan NIM, lalu meminta NIM yang ingin dicari. Pencarian dilakukan dengan dua metode, yaitu sequential search dan binary search. Sequential search mengecek data satu per satu dari awal sampai akhir, sedangkan binary search mencari data dengan membagi area pencarian menjadi dua bagian sehingga lebih cepat, tetapi syaratnya data harus sudah terurut berdasarkan

NIM. Jika data ditemukan, program menampilkan indeks, nama, NIM, dan IPK mahasiswa. Jika tidak ditemukan, program menampilkan pesan bahwa data tidak ada.

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Jawaban :

```
package main

import "fmt"

func main() {
    var suara int
    var masuk, sah int
    var hitung [21]int

    for {
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }

        masuk++

        if suara >= 1 && suara <= 20 {
            sah++
            hitung[suara]++
        }
    }
}
```

```

fmt.Println("Suara masuk:", masuk)
fmt.Println("Suara sah:", sah)

for i := 1; i <= 20; i++ {
    if hitung[i] > 0 {
        fmt.Printf("%d: %d\n", i, hitung[i])
    }
}
}

```

Output :

```

7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2

```

Penjelasan :

Program ini membaca data suara pemilihan ketua RT sampai ditemukan angka 0. Setiap suara yang terbaca dihitung sebagai suara masuk, sedangkan suara yang bernilai 1 sampai 20 dihitung sebagai suara sah. Program kemudian menghitung jumlah suara tiap calon dan menampilkan hanya calon yang memperoleh suara.

2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Jawaban :

```
package main

import "fmt"

func main() {
    var suara int
    var masuk, sah int
    var hitung [21]int

    for {
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }

        masuk++

        if suara >= 1 && suara <= 20 {
            sah++
            hitung[suara]++
        }
    }

    ketua := 0
    wakil := 0

    for i := 1; i <= 20; i++ {
        if hitung[i] > hitung[ketua] {
            wakil = ketua
            ketua = i
        } else if hitung[i] > hitung[wakil] && i != ketua {
            wakil = i
        }
    }

    fmt.Println("Suara masuk:", masuk)
    fmt.Println("Suara sah:", sah)
    fmt.Println("Ketua RT:", ketua)
    fmt.Println("Wakil ketua:", wakil)
}
```

Output :

```
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
```

Penjelasan :

Program ini membaca data suara pemilihan ketua RT hingga input bernilai 0. Data valid adalah angka 1 sampai 20, sedangkan data di luar itu tidak dihitung sebagai suara sah. Setelah semua suara diproses, program menentukan calon dengan suara terbanyak sebagai ketua RT dan calon dengan suara terbanyak kedua sebagai wakil ketua RT. Jika jumlah suara sama, calon dengan nomor lebih kecil dipilih lebih dahulu.

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    kiri := 0
```

```

    kanan := n - 1

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if data[tengah] == k {
            return tengah
        } else if data[tengah] < k {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }

    return -1
}

func main() {
    var n, k int

    fmt.Scan(&n, &k)
    isiArray(n)

    hasil := posisi(n, k)

    if hasil != -1 {
        fmt.Println(hasil)
    } else {
        fmt.Println("TIDAK ADA")
    }
}

```

Output :

```

12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\COLORFUL\Documents\ALPROTELKOM\SEMESTER 2> go run
Wijaya\Week11\Unguided\Unguided3\Unguided3.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA

```

Penjelasan :

Program ini digunakan untuk mencari posisi bilangan k dalam kumpulan data integer positif yang sudah terurut membesar. Program membaca nilai n dan k, lalu membaca sebanyak n data ke dalam array. Pencarian dilakukan menggunakan binary search agar lebih cepat. Jika data ditemukan, program menampilkan indeksnya dimulai dari 0; jika tidak ditemukan, program menampilkan TIDAK ADA.

D. Kesimpulan

Praktikum ini memberikan pemahaman mendalam tentang dua algoritma pencarian fundamental yaitu sequential search dan binary search beserta penerapannya dalam bahasa pemrograman Go. Sequential search terbukti sederhana dalam implementasi dan dapat digunakan pada data yang tidak terurut, namun memiliki kompleksitas waktu $O(n)$ yang kurang efisien untuk data besar. Sebaliknya, binary search menawarkan efisiensi tinggi dengan kompleksitas $O(\log n)$ tetapi memerlukan syarat data harus terurut terlebih dahulu. Melalui berbagai program yang dibuat, mulai dari pencarian sederhana pada array binatang, pencarian bilangan dalam data terurut, hingga pencarian data mahasiswa menggunakan struct, praktikum ini menunjukkan fleksibilitas dan aplikasi nyata dari kedua algoritma dalam menyelesaikan berbagai permasalahan pencarian data.

Praktikum ini juga menekankan pentingnya pemilihan struktur data yang tepat dan penerapan defensive programming dalam pengembangan aplikasi. Penggunaan struct untuk mengelompokkan data mahasiswa yang saling berkaitan seperti nama, NIM, dan IPK membuat program lebih terorganisir dan mudah dipelihara. Implementasi kedua algoritma pada program yang sama memungkinkan perbandingan langsung performa keduanya, meskipun pada data kecil perbedaan

tidak terlalu signifikan. Penerapan validasi input dan penanganan kondisi data tidak ditemukan menunjukkan pentingnya membangun aplikasi yang robust dan user-friendly, sehingga program dapat menangani berbagai skenario input dengan baik.

Praktikum ini mengajarkan bahwa pemilihan algoritma harus disesuaikan dengan karakteristik data dan kebutuhan aplikasi. Sequential search lebih cocok untuk data kecil atau tidak terurut dimana kesederhanaan implementasi menjadi prioritas, sementara binary search ideal untuk data besar yang sudah terurut dan sering dilakukan pencarian. Pemahaman tentang trade-off antara kompleksitas implementasi, efisiensi waktu, dan kondisi data merupakan keterampilan penting yang harus dimiliki programmer untuk mengembangkan perangkat lunak yang optimal dan efisien sesuai dengan konteks permasalahan yang dihadapi.